Software Engineering Project May-2026

Milestone 4 Deliverables for ZenStudy

Submitted By

Team Name: DeerInTheCode

Team Code : Team-051



IITM Online BS Degree Program,
Indian Institute of Technology, Madras,
Chennai Tamil Nadu, India, 600036

Community Services Platform for Library Management

Project Details

Software Name: ZenStudy

Department Context: Library Management System

System Type: Small Business Logistics & Community Services Platform

Date of Submission: 11-08-2026

Team Details

Team Name: DeerInTheCode
Team Code: Team-051

Name	Roll Number	Role
DIKSHA SHARMA	23f2004899	Product Manager
KARTHIK V N	23f1000033	Frontend Developer
NEHA CHACKO	24f1002416	Scrum Master
GAURAV TOMAR	23f1000805	Backend Developer
AKSHATHA A	23f1001878	Code Reviewer/Tester

API TEST CASES

Test cases which were implemented as part of milestone-3

API besing tested	Inputs	Expected output	Actual output	Result
POST /api/auth/login	JSON: {"username": "admin", "password": "test-password"}	HTTP 200; response contains access_token	HTTP 200; access_token present	**Success**
POST /api/auth/login	JSON: {"username": "admin", "password": "wrongpassword"}	HTTP 401; message "Invalid username or password"	HTTP 401; message matches	**Success**
GET /api/admin/staff	No Authorization header	HTTP 401	HTTP 401	**Success**
GET /api/admin/staff	Bearer admin JWT	HTTP 200; JSON array of staff	HTTP 200; list returned	**Success**
POST /api/admin/staff	Bearer admin; JSON: {"username", "password"} only (no email)	HTTP 400	HTTP 400	**Success**
POST /api/admin/staff	Bearer admin; JSON: username, password, email, role	HTTP 200 or 201; staff created	HTTP 201; staff created	**Success**
GET /api/admin/membership-plans	Bearer admin JWT	HTTP 200; JSON array of plans	HTTP 200; list returned	**Success**
GET /api/admin/settings	Bearer admin JWT	HTTP 200; settings object	HTTP 200	**Success**
GET /api/admin/seats	Bearer admin JWT	HTTP 200; seats list	HTTP 200	**Success**
GET /api/profile	Bearer admin JWT	HTTP 200; username == "admin"	HTTP 200; username admin	**Success**
POST /api/staff/students/register	Bearer admin; username, password, email, full_name	HTTP 201; response contains student	HTTP 201; student present	**Success**
GET /api/staff/students/999	Bearer admin JWT	HTTP 404	HTTP 404	**Success**
GET /api/staff/students/999/verify	Bearer admin JWT	HTTP 404	HTTP 404	**Success**

POST /api/students/me/bookings/{id}/extension-request	Bearer student; JSON: {"requested_end_time": "11:30"}	HTTP 201; extension_request.status == "Pending"	HTTP 201; status Pending	**Success**
POST /api/students/me/bookings/{id}/extension-request	Bearer student; {"requested_end_time": "not-a-time"}	HTTP 400	HTTP 400	**Success**
POST /api/students/me/bookings/{id}/extension-request	Bearer student; {"requested_end_time": "10:30"} (before current end)	HTTP 400	HTTP 400	**Success**
POST /api/students/me/bookings/{id}/extension-request	Bearer student; second request while first still Pending	HTTP 409	HTTP 409	**Success**
POST /api/students/me/bookings/{id}/extension-request	Bearer other student; another student's booking_id	HTTP 404	HTTP 404	**Success**
GET /api/staff/extension-requests	Bearer staff; after student creates pending request	HTTP 200; total == 1; seat_number present	HTTP 200; total 1	**Success**
POST /api/staff/extension-requests/{id}/approve	Bearer staff; no seat clash	HTTP 200; booking.end_time == "11:30"; same seat_id	HTTP 200; extended on same seat	**Success**
POST /api/staff/extension-requests/{id}/approve	Bearer staff; clash then approve with seat_id=2	First call HTTP 409; then HTTP 200 with seat_id 2	HTTP 409 then HTTP 200; seat reassigned	**Success**
POST /api/staff/extension-requests/{id}/reject	Bearer staff; pending extension_request_id	HTTP 200; status == "Rejected"	HTTP 200; Rejected	**Success**
POST /api/staff/extension-requests/{id}/approve	Bearer staff; already rejected request	HTTP 400	HTTP 400	**Success**
POST extension-request / GET extension-requests	Staff hits student route; student hits staff route	HTTP 403 for both	HTTP 403 for both	**Success**
POST /api/students/me/bookings/{id}/extension-request	extension_limit_minutes=15; request 30min then 10min	First HTTP 400; second HTTP 201	HTTP 400 then HTTP 201	**Success**
POST /api/staff/checkin	Bearer staff; non-JSON body	HTTP 400	HTTP 400	**Success**
POST /api/staff/checkin	Bearer staff; JSON {}	HTTP 400	HTTP 400	**Success**
POST /api/staff/checkin	Bearer staff; {"checkin_code": "NOPE1234"}	HTTP 404	HTTP 404	**Success**

POST /api/staff/checkin	Bearer staff; valid checkin_code for today's booking	HTTP 201; attendance created; booking status "Checked-In"	HTTP 201; Checked-In	**Success**
POST /api/staff/checkin	Bearer staff; same checkin_code twice	Second call HTTP 400	HTTP 400	**Success**
POST /api/staff/checkin	Bearer staff; checkin_code for future-dated booking	HTTP 400	HTTP 400	**Success**
POST /api/staff/checkin	Bearer staff; booking already cancelled	HTTP 400	HTTP 400	**Success**
POST /api/staff/checkin	Bearer student JWT	HTTP 403	HTTP 403	**Success**
GET /api/staff/bookings/no-show-candidates	Bearer staff; overdue Confirmed booking exists	HTTP 200; overdue booking_id in candidates	HTTP 200; candidate listed	**Success**
GET /api/staff/bookings/no-show-candidates	Bearer staff; future booking exists	HTTP 200; future booking_id not in candidates	HTTP 200; future excluded	**Success**
POST /api/staff/bookings/{id}/no-show	Bearer staff; overdue booking_id	HTTP 200; booking.status == "No-Show"	HTTP 200; No-Show	**Success**
POST /api/staff/bookings/{id}/no-show then POST /api/bookings	Mark no-show then rebook same seat/slot	No-show HTTP 200; new booking HTTP 201	Seat freed; rebook HTTP 201	**Success**
POST /api/staff/bookings/{id}/no-show	Bearer staff; grace period set very high	HTTP 400	HTTP 400	**Success**
POST /api/staff/bookings/9999/no-show	Bearer staff	HTTP 404	HTTP 404	**Success**
POST /api/staff/bookings/{id}/no-show	Bearer staff; cancelled booking	HTTP 400	HTTP 400	**Success**
GET no-show-candidates / POST .../no-show	Bearer student	HTTP 403 for both	HTTP 403 for both	**Success**
POST /api/bookings/{id}/checkout	Bearer student; Checked-In booking owned by student	HTTP 200; status "Completed"; checkout_time set	HTTP 200; Completed	**Success**
POST /api/bookings/{id}/checkout	Bearer staff; Checked-In booking	HTTP 200	HTTP 200	**Success**
POST /api/bookings/{id}/checkout	Bearer student; Confirmed (not Checked-In) booking	HTTP 400	HTTP 400	**Success**
POST /api/bookings/9999/checkout	Bearer student	HTTP 404	HTTP 404	**Success**

POST /api/bookings/{id}/checkout	Bearer other student	HTTP 403	HTTP 403	**Success**
POST /api/bookings/{id}/checkout	Bearer student; checkout twice	Second call HTTP 400	HTTP 400	**Success**
POST /api/staff/payments	Bearer staff; student_id, amount=500, payment_mode="Cash"	HTTP 201; payment fields match	HTTP 201; amount 500 Cash	**Success**
POST /api/staff/payments	payment_mode="Cheque"	HTTP 400	HTTP 400	**Success**
POST /api/staff/payments	No amount field	HTTP 400	HTTP 400	**Success**
POST /api/staff/payments	amount=-50	HTTP 400	HTTP 400	**Success**
POST /api/staff/payments	student_id=9999	HTTP 404	HTTP 404	**Success**
POST /api/staff/payments	Bearer student	HTTP 403	HTTP 403	**Success**
GET /api/students/me/payments	Bearer student; after staff records payment	HTTP 200; total_payments == 1	HTTP 200; 1 payment	**Success**
GET /api/students/me/payments	Bearer student with no payments	HTTP 200; total_payments == 0	HTTP 200; empty list	**Success**
GET /api/staff/students/{id}/payments	Bearer staff; student with 2 payments	HTTP 200; total_payments == 2	HTTP 200; 2 payments	**Success**
GET /api/staff/students/9999/payments	Bearer staff	HTTP 404	HTTP 404	**Success**
GET /api/students/me/payments	Bearer staff	HTTP 403	HTTP 403	**Success**
GET /api/staff/students/{id}/payments	Bearer student	HTTP 403	HTTP 403	**Success**
POST /api/students/me/complaints	Bearer student; title + description	HTTP 201; status "Open"	HTTP 201; Open	**Success**
POST /api/students/me/complaints	description only	HTTP 400	HTTP 400	**Success**
POST /api/students/me/complaints	title only	HTTP 400	HTTP 400	**Success**
POST /api/students/me/complaints	Non-JSON body	HTTP 400	HTTP 400	**Success**
POST /api/students/me/complaints	Bearer staff	HTTP 403	HTTP 403	**Success**

Test cases which were implemented as part of milestone-4

Test Case 1

- **API being tested**: POST /api/admin/membership-plans
- **Inputs**: Bearer admin; plan_name="Daily", duration_days=1, price=50
- **Expected Output**: HTTP 201
- **Actual Output**: HTTP 201
- **Result**: Success

Snapshot of pytest function

```python
def test_create_membership_plan_success(client, admin_token):
    """Admin can create a new membership plan"""
    headers = {"Authorization": f"Bearer {admin_token}"}
    response = client.post("/api/admin/membership-plans", json={
        "plan_name": "Daily", "duration_days": 1, "price": 50
    }, headers=headers)
    assert response.status_code == 201
    assert response.get_json()["plan_name"] == "Daily"
```

Test Case 2

- **API being tested**: PUT /api/admin/membership-plans/{plan_id}
- **Inputs**: Bearer admin; price=150
- **Expected Output**: HTTP 200
- **Actual Output**: HTTP 200
- **Result**: Success

Snapshot of pytest function

```python
def test_update_membership_plan_success(client, admin_token):
    """Admin can update an existing membership plan"""
    headers = {"Authorization": f"Bearer {admin_token}"}
    plans = client.get("/api/admin/membership-plans", headers=headers).get_json()
    plan_id = plans[0]["plan_id"]
    response = client.put(f"/api/admin/membership-plans/{plan_id}", json={
        "price": 150
    }, headers=headers)
    assert response.status_code == 200
    assert response.get_json()["price"] == 150.0
```

Test Case 3

- **API being tested**: DELETE /api/admin/membership-plans/{plan_id}
- **Inputs**: Bearer admin; active plan
- **Expected Output**: HTTP 200
- **Actual Output**: HTTP 200
- **Result**: Success

Snapshot of pytest function

```python
def test_retire_membership_plan_success(client, admin_token):
    """Admin can retire (soft-delete) a membership plan"""
    headers = {"Authorization": f"Bearer {admin_token}"}
    created = client.post("/api/admin/membership-plans", json={
        "plan_name": "Temp Plan", "duration_days": 3, "price": 75
    }, headers=headers).get_json()
    response = client.delete(f"/api/admin/membership-plans/{created['plan_id']}", headers=headers)
    assert response.status_code == 200
    assert response.get_json()["plan"]["is_active"] is False
```

Test Case 4

- **API being tested**: GET /api/admin/memberships (?status=Active)
- **Inputs**: Bearer admin
- **Expected Output**: HTTP 200
- **Actual Output**: HTTP 200
- **Result**: Success

Snapshot of pytest function

```python
def test_get_memberships_list(client, admin_token):
    """Admin can list all memberships, optionally filtered by status"""
    headers = {"Authorization": f"Bearer {admin_token}"}
    response = client.get("/api/admin/memberships", headers=headers)
    assert response.status_code == 200
    assert isinstance(response.get_json(), list)

    filtered = client.get("/api/admin/memberships?status=Active", headers=headers)
    assert filtered.status_code == 200
    for s in filtered.get_json():
        assert s["membership_status"] == "Active"
```

Test Case 5

- **API being tested**: PUT /api/admin/memberships/{id}
- **Inputs**: Bearer admin; {"extend_days": 30}
- **Expected Output**: HTTP 200
- **Actual Output**: HTTP 200
- **Result**: Success

Snapshot of pytest function

```python
def test_update_membership_extend(client, admin_token):
    """Admin can extend a student's membership expiry"""
    student = _register_pending_student(client, admin_token, "extendstudent")
    headers = {"Authorization": f"Bearer {admin_token}"}
    client.post(f"/api/admin/memberships/{student['student_id']}/approve", headers=headers)
    response = client.put(f"/api/admin/memberships/{student['student_id']}", json={
        "extend_days": 30
    }, headers=headers)
    assert response.status_code == 200
    assert response.get_json()["student"]["membership_expiry"] is not None
```

Test Case 6

- **API being tested**: PUT /api/admin/settings
- **Inputs**: Bearer admin; open_time="08:00", close_time="22:00"
- **Expected Output**: HTTP 200; hours updated
- **Actual Output**: HTTP 200; hours updated
- **Result**: Success

Snapshot of pytest function

```
def test_update_library_settings_success(client, admin_token):
    """Admin can update library operating hours"""
    headers = {"Authorization": f"Bearer {admin_token}"}
    response = client.put("/api/admin/settings", json={
        "open_time": "08:00", "close_time": "22:00"
    }, headers=headers)
    assert response.status_code == 200
    settings = response.get_json()["settings"]
    assert settings["open_time"] == "08:00"
    assert settings["close_time"] == "22:00"
```

Test Case 7

- **API being tested**: POST /api/admin/seats
- **Inputs**: Bearer admin; seat_number="S99", seat_type="Premium"
- **Expected Output**: HTTP 201
- **Actual Output**: HTTP 201
- **Result**: Success

Snapshot of pytest function

```
def test_create_seat_success(client, admin_token):
    """Admin can add a new seat to the inventory"""
    headers = {"Authorization": f"Bearer {admin_token}"}
    response = client.post("/api/admin/seats", json={
        "seat_number": "S99", "seat_type": "Premium", "is_active": True
    }, headers=headers)
    assert response.status_code == 201
    assert response.get_json()["seat_number"] == "S99"
```

Test Case 8

- **API being tested:** GET /api/seats/availability
- **Inputs**: Bearer student; date, start_time, end_time query params
- **Expected Output**: HTTP 200
- **Actual Output**: HTTP 200
- **Result**: Success

Snapshot of pytest function

```python
def test_seat_availability_success(client, student_token):
    """Authenticated user can search for available seats in a time slot"""
    headers = {"Authorization": f"Bearer {student_token}"}
    response = client.get(
        "/api/seats/availability?date=2026-12-01&start_time=10:00&end_time=11:00",
        headers=headers,
    )
    assert response.status_code == 200
    body = response.get_json()
    assert body["total_available"] > 0
    assert len(body["available_seats"]) == body["total_available"]
```

Test Case 9

- **API being tested**: POST /api/bookings
- **Inputs**: Bearer student; seat_id, date, start_time, end_time
- **Expected Output**: HTTP 201; status=" Confirmed"
- **Actual Output**: HTTP 201; status=" Confirmed"
- **Result**: Success

Snapshot of pytest function

```python
def test_create_booking_success(client, student_token):
    """Student can create a booking for an available seat"""
    headers = {"Authorization": f"Bearer {student_token}"}
    response = client.post("/api/bookings", json={
        "seat_id": 3, "date": "2026-10-01", "start_time": "14:00", "end_time": "15:00"
    }, headers=headers)
    assert response.status_code == 201
    assert response.get_json()["booking"]["status"] == "Confirmed"
```

Test Case 10

- **API being tested**: PUT /api/bookings/{id}
- **Inputs**: Bearer student; start_time="10:30", end_time="11:30"
- **Expected Output**: HTTP 200; start_time updated
- **Actual Output**: HTTP 200; start_time updated
- **Result**: Success

Snapshot of pytest function

```python
def test_modify_booking_success(client, student_token, confirmed_booking):
    """Student can modify their confirmed booking's time"""
    headers = {"Authorization": f"Bearer {student_token}"}
    response = client.put(f"/api/bookings/{confirmed_booking['booking_id']}", json={
        "start_time": "10:30", "end_time": "11:30"
    }, headers=headers)
    assert response.status_code == 200
    assert response.get_json()["booking"]["start_time"] == "10:30"
```

Test Case 11

- **API being tested**: POST /api/bookings/{id}/cancel
- **Inputs**: Bearer student; own Confirmed booking
- **Expected Output**: HTTP 200; status= ”Cancelled”
- **Actual Output**: HTTP 200; status=” Cancelled”
- **Result**: Success

Snapshot of pytest function

```
def test_cancel_booking_success(client, student_token, confirmed_booking):
    """Student can cancel their own confirmed booking"""
    headers = {"Authorization": f"Bearer {student_token}"}
    response = client.post(f"/api/bookings/{confirmed_booking['booking_id']}/cancel", headers=headers)
    assert response.status_code == 200
    assert response.get_json()["booking"]["status"] == "Cancelled"
```

Test Case 12

- **API being tested**: GET /api/students/me/bookings
- **Inputs**: Bearer student; has bookings
- **Expected Output**: HTTP 200
- **Actual Output**: HTTP 200
- **Result**: Success

Snapshot of pytest function

```
def test_student_booking_history(client, student_token, confirmed_booking):
    """Student can view their booking history"""
    headers = {"Authorization": f"Bearer {student_token}"}
    response = client.get("/api/students/me/bookings", headers=headers)
    assert response.status_code == 200
    body = response.get_json()
    assert body["total_bookings"] >= 1
    ids = [b["booking_id"] for b in body["bookings"]]
    assert confirmed_booking["booking_id"] in ids
```

Test Case 13

- **API being tested**: GET /api/staff/bookings/today
- **Inputs**: Bearer staff; today's Confirmed booking exists
- **Expected Output**: HTTP 200
- **Actual Output**: HTTP 200
- **Result**: Success

Snapshot of pytest function

```python
def test_staff_today_bookings(client, staff_token, todays_booking):
    """Staff can list today's confirmed bookings"""
    headers = {"Authorization": f"Bearer {staff_token}"}
    response = client.get("/api/staff/bookings/today", headers=headers)
    assert response.status_code == 200
    body = response.get_json()
    assert body["total_expected"] >= 1
    ids = [b["booking_id"] for b in body["bookings"]]
    assert todays_booking["booking_id"] in ids
```

Test Case 14

- **API being tested**: POST /api/staff/bookings/walk-in
- **Inputs**: Bearer staff; active student_id, duration_minutes=60
- **Expected Output**: HTTP 201; status= “Confirmed”
- **Actual Output**: HTTP 201; status= “Confirmed”
- **Result**: Success

Snapshot of pytest function

```python
def test_walk_in_booking_success(client, staff_token, admin_token):
    """Staff can create a walk-in booking for an active student"""
    student = _register_pending_student(client, admin_token, "walkinstudent")
    headers = {"Authorization": f"Bearer {admin_token}"}
    client.post(f"/api/admin/memberships/{student['student_id']}/approve", headers=headers)

    staff_headers = {"Authorization": f"Bearer {staff_token}"}
    response = client.post("/api/staff/bookings/walk-in", json={
        "student_id": student["student_id"], "duration_minutes": 60
    }, headers=staff_headers)
    assert response.status_code == 201
    assert response.get_json()["booking"]["status"] == "Confirmed"
```

Test Case 15

- **API being tested**: GET /api/staff/complaints
- **Inputs**: Bearer staff; after student submits complaint
- **Expected Output**: HTTP 200
- **Actual Output**: HTTP 200
- **Result**: Success

Snapshot of pytest function

```python
def test_staff_list_complaints(client, staff_token, student_token):
    """Staff can list all submitted complaints"""
    student_headers = {"Authorization": f"Bearer {student_token}"}
    client.post("/api/students/me/complaints", json={
        "title": "Broken chair", "description": "Seat S05 is broken"
    }, headers=student_headers)

    staff_headers = {"Authorization": f"Bearer {staff_token}"}
    response = client.get("/api/staff/complaints", headers=staff_headers)
    assert response.status_code == 200
    assert response.get_json()["total_complaints"] >= 1
```

Test Case 16

- **API being tested**: PUT /api/staff/complaints/{id}
- **Inputs**: Bearer staff; {"status": "InProgress"}
- **Expected Output**: HTTP 200; status= “InProgress”
- **Actual Output**: HTTP 200; status= “InProgress”
- **Result**: Success

Snapshot of pytest function

```python
def test_staff_update_complaint_status(client, staff_token, student_token):
    """Staff can update a complaint's status"""
    student_headers = {"Authorization": f"Bearer {student_token}"}
    complaint = client.post("/api/students/me/complaints", json={
        "title": "AC issue", "description": "Too cold"
    }, headers=student_headers).get_json()["complaint"]

    staff_headers = {"Authorization": f"Bearer {staff_token}"}
    response = client.put(f"/api/staff/complaints/{complaint['complaint_id']}", json={
        "status": "InProgress"
    }, headers=staff_headers)
    assert response.status_code == 200
    assert response.get_json()["complaint"]["status"] == "InProgress"
```

Test Case 17

- **API being tested**: POST /api/admin/announcements
- **Inputs**: Bearer admin; title + content
- **Expected Output**: HTTP 201; announcement created
- **Actual Output**: HTTP 201; announcement created
- **Result**: Success

Snapshot of pytest function

```python
def test_admin_create_announcement(client, admin_token):
    """Admin can create a new announcement"""
    headers = {"Authorization": f"Bearer {admin_token}"}
    response = client.post("/api/admin/announcements", json={
        "title": "Holiday Hours", "content": "Library closed on Sunday"
    }, headers=headers)
    assert response.status_code == 201
    assert response.get_json()["announcement"]["title"] == "Holiday Hours"
```

Test Case 18

- **API being tested**: GET /api/announcements
- **Inputs**: Bearer admin; active announcement exists
- **Expected Output**: HTTP 200
- **Actual Output**: HTTP 200
- **Result**: Success

Snapshot of pytest function

```python
def test_list_announcements(client, admin_token):
    """Any authenticated user can list active announcements"""
    headers = {"Authorization": f"Bearer {admin_token}"}
    client.post("/api/admin/announcements", json={
        "title": "Test Ann", "content": "Test content"
    }, headers=headers)
    response = client.get("/api/announcements", headers=headers)
    assert response.status_code == 200
    assert isinstance(response.get_json(), list)
    assert len(response.get_json()) >= 1
```

Test Case 19

- **API being tested**: GET /api/admin/reports/occupancy
- **Inputs**: Bearer admin
- **Expected Output**: HTTP 200; occupancy + start_date keys
- **Actual Output**: HTTP 200; occupancy + start_date keys
- **Result**: Success

Snapshot of pytest function

```python
def test_occupancy_report(client, admin_token):
    """Admin can retrieve the occupancy report"""
    headers = {"Authorization": f"Bearer {admin_token}"}
    response = client.get("/api/admin/reports/occupancy", headers=headers)
    assert response.status_code == 200
    body = response.get_json()
    assert "occupancy" in body
    assert "start_date" in body
```

Test Case 20

- **API being tested**: GET /api/admin/reports/membership-stats
- **Inputs**: Bearer admin
- **Expected Output**: HTTP 200; total_students + by_status
- **Actual Output**: HTTP 200; total_students + by_status
- **Result**: Success

Snapshot of pytest function

```python
def test_membership_stats_report(client, admin_token):
    """Admin can retrieve membership statistics"""
    headers = {"Authorization": f"Bearer {admin_token}"}
    response = client.get("/api/admin/reports/membership-stats", headers=headers)
    assert response.status_code == 200
    body = response.get_json()
    assert "total_students" in body
    assert "by_status" in body
```

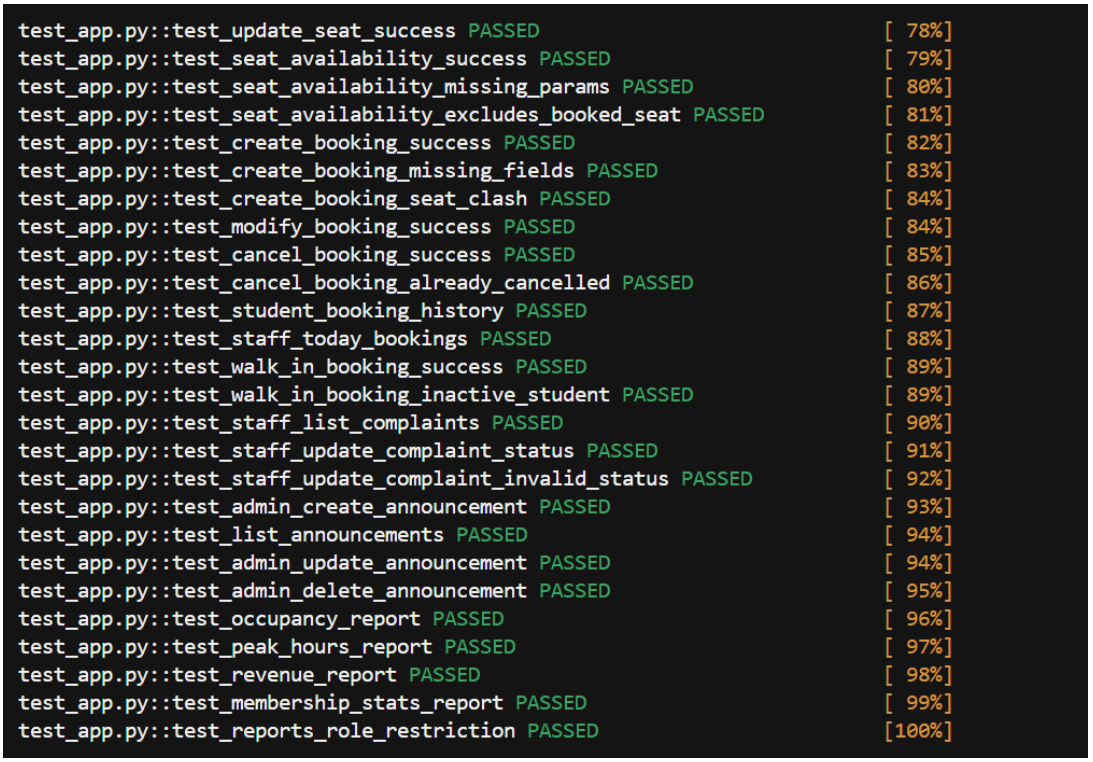
Consolidated Report

```
============================= test session starts =============================
platform win32 -- Python 3.13.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Python313\python.exe
cachedir: .pytest_cache
rootdir: D:\IITM\se\milestone-4\MAY2026-Team-051
plugins: anyio-4.11.0, langsmith-0.10.10
collected 119 items

test_app.py::test_login_success PASSED                                   [  0%]
test_app.py::test_login_failure PASSED                                   [  1%]
test_app.py::test_get_staff_list_unauthorized PASSED                     [  2%]
test_app.py::test_get_staff_list_authorized PASSED                       [  3%]
test_app.py::test_create_staff_requires_complete_details PASSED          [  4%]
test_app.py::test_create_staff_success PASSED                            [  5%]
test_app.py::test_get_membership_plans PASSED                            [  5%]
test_app.py::test_get_library_settings PASSED                            [  6%]
test_app.py::test_get_seats PASSED                                       [  7%]
test_app.py::test_get_profile PASSED                                     [  8%]
test_app.py::test_staff_register_student PASSED                          [  9%]
test_app.py::test_staff_student_detail_not_found PASSED                  [ 10%]
test_app.py::test_staff_verify_student_not_found PASSED                  [ 10%]
test_app.py::test_student_request_extension_success PASSED               [ 11%]
test_app.py::test_student_request_extension_invalid_time_format PASSED   [ 12%]
test_app.py::test_student_request_extension_before_current_end_rejected PASSED [ 13%]
test_app.py::test_student_request_extension_duplicate_pending_conflict PASSED [ 14%]
test_app.py::test_student_cannot_request_extension_on_others_booking PASSED [ 15%]
test_app.py::test_staff_view_pending_extension_requests PASSED           [ 15%]
test_app.py::test_staff_approve_extension_extends_same_seat PASSED       [ 16%]
test_app.py::test_staff_approve_extension_reassigns_seat_on_clash PASSED [ 17%]
test_app.py::test_staff_reject_extension PASSED                          [ 18%]
test_app.py::test_staff_cannot_resolve_already_resolved_extension PASSED [ 19%]
test_app.py::test_extension_endpoints_role_restrictions PASSED           [ 20%]
test_app.py::test_extension_limit_minutes_enforced PASSED                [ 21%]
test_app.py::test_staff_checkin_missing_body PASSED                      [ 21%]
```

```
test_app.py::test_staff_checkin_missing_code PASSED                    [ 22%]
test_app.py::test_staff_checkin_invalid_code PASSED                    [ 23%]
test_app.py::test_staff_checkin_success PASSED                         [ 24%]
test_app.py::test_staff_checkin_already_checked_in PASSED              [ 25%]
test_app.py::test_staff_checkin_wrong_date_rejected PASSED             [ 26%]
test_app.py::test_staff_checkin_cancelled_booking_rejected PASSED      [ 26%]
test_app.py::test_staff_checkin_role_restriction PASSED                [ 27%]
test_app.py::test_no_show_candidates_lists_overdue_booking PASSED      [ 28%]
test_app.py::test_no_show_candidates_excludes_future_booking PASSED    [ 29%]
test_app.py::test_mark_no_show_success PASSED                          [ 30%]
test_app.py::test_mark_no_show_frees_seat_for_new_booking PASSED       [ 31%]
test_app.py::test_mark_no_show_before_grace_period_rejected PASSED     [ 31%]
test_app.py::test_mark_no_show_not_found PASSED                        [ 32%]
test_app.py::test_mark_no_show_already_cancelled_rejected PASSED       [ 33%]
test_app.py::test_no_show_endpoints_role_restriction PASSED            [ 34%]
test_app.py::test_checkout_success PASSED                              [ 35%]
test_app.py::test_checkout_by_staff_allowed PASSED                     [ 36%]
test_app.py::test_checkout_not_checked_in_rejected PASSED              [ 36%]
test_app.py::test_checkout_not_found PASSED                            [ 37%]
test_app.py::test_checkout_not_owner_rejected PASSED                   [ 38%]
test_app.py::test_checkout_twice_rejected PASSED                       [ 39%]
test_app.py::test_staff_record_payment_success PASSED                  [ 40%]
test_app.py::test_staff_record_payment_invalid_mode PASSED             [ 41%]
test_app.py::test_staff_record_payment_missing_amount PASSED           [ 42%]
test_app.py::test_staff_record_payment_negative_amount PASSED          [ 42%]
test_app.py::test_staff_record_payment_student_not_found PASSED        [ 43%]
test_app.py::test_staff_record_payment_role_restriction PASSED         [ 44%]
test_app.py::test_student_views_own_payment_history PASSED             [ 45%]
test_app.py::test_student_payment_history_empty_when_no_payments PASSED [ 46%]
test_app.py::test_staff_views_student_payment_history PASSED           [ 47%]
test_app.py::test_staff_payment_history_student_not_found PASSED       [ 47%]
test_app.py::test_student_payment_history_role_restriction PASSED      [ 48%]
test_app.py::test_staff_payment_history_role_restriction PASSED        [ 49%]
```

```
test_app.py::test_student_submit_complaint_success PASSED              [ 50%]
test_app.py::test_student_submit_complaint_missing_title PASSED        [ 51%]
test_app.py::test_student_submit_complaint_missing_description PASSED  [ 52%]
test_app.py::test_student_submit_complaint_missing_body PASSED         [ 52%]
test_app.py::test_student_submit_complaint_role_restriction PASSED     [ 53%]
test_app.py::test_reset_password_success PASSED                        [ 54%]
test_app.py::test_reset_password_wrong_old_password PASSED             [ 55%]
test_app.py::test_reset_password_missing_fields PASSED                 [ 56%]
test_app.py::test_update_profile_success PASSED                        [ 57%]
test_app.py::test_update_profile_student_forbidden PASSED              [ 57%]
test_app.py::test_student_me_get PASSED                                [ 58%]
test_app.py::test_student_me_put PASSED                                [ 59%]
test_app.py::test_student_me_role_restriction PASSED                   [ 60%]
test_app.py::test_get_staff_detail_success PASSED                      [ 61%]
test_app.py::test_get_staff_detail_not_found PASSED                    [ 62%]
test_app.py::test_update_staff_success PASSED                          [ 63%]
test_app.py::test_deactivate_staff_success PASSED                      [ 63%]
test_app.py::test_deactivate_staff_already_deactivated PASSED          [ 64%]
test_app.py::test_create_membership_plan_success PASSED                [ 65%]
test_app.py::test_create_membership_plan_missing_name PASSED           [ 66%]
test_app.py::test_update_membership_plan_success PASSED                [ 67%]
test_app.py::test_retire_membership_plan_success PASSED                [ 68%]
test_app.py::test_retire_membership_plan_already_retired PASSED        [ 68%]
test_app.py::test_get_pending_memberships PASSED                       [ 69%]
test_app.py::test_get_memberships_list PASSED                          [ 70%]
test_app.py::test_reject_membership_success PASSED                     [ 71%]
test_app.py::test_disapprove_membership_success PASSED                 [ 72%]
test_app.py::test_update_membership_extend PASSED                      [ 73%]
test_app.py::test_staff_student_detail_success PASSED                  [ 73%]
test_app.py::test_staff_verify_student_active PASSED                   [ 74%]
test_app.py::test_update_library_settings_success PASSED               [ 75%]
test_app.py::test_update_library_settings_invalid_hours PASSED         [ 76%]
test_app.py::test_create_seat_success PASSED                           [ 77%]
```



PLANNED IMPROVEMENTS

In the upcoming sprint, we plan to focus on the following:

- Integrating the frontend with the backend APIs.
- Refining the error handling to provide more detailed validation errors.